

Minimum Enclosing Circle

Consider the following problem:

Library Checker - Minimum Enclosing Circle

You're given $n \leq 10^5$ points $p_i = (x_i, y_i)$.

For each p_i , find whether it lies on the circumference of the minimum enclosing circle of $\{p_1, \dots, p_n\}$.

Here, by the minimum enclosing circle (MEC) we mean a circle with minimum possible radius that contains all the n points, inside the circle or on its boundary. This problem has a simple randomized solution that, on first glance, looks like it would run in $O(n^3)$, but actually works in $O(n)$ expected time.

To better understand the reasoning below, we should immediately note that the solution to the problem is unique:

Why is the MEC unique?

Consider the following setup: Let r be the radius of the MEC. We draw a circle of radius r around each of the points $p_1, \dots, p_n$. Geometrically, the centers of circles that have radius r and cover all the points $p_1, \dots, p_n$ form the intersection of all n circles.

Now, if the intersection is just a single point, this already proves that it is unique. Otherwise, the intersection is a shape of non-zero area, so we can reduce r by a tiny bit, and still have non-empty intersection, which contradicts the assumption that r was the minimum possible radius of the enclosing circle.

With a similar logic, we can also show the uniqueness of the MEC if we additionally demand that it passes through a given specific point p_i or two points p_i and p_j (it is also unique because its radius uniquely defines it).

Alternatively, we can also assume that there are two MECs, and then notice that their intersection (which contains the points $p_1, \dots, p_n$ already) must have a smaller diameter than initial circles, and thus can be covered with a smaller circle.

Welzl's algorithm

For brevity, let's denote $\text{mec}(p_1, \dots, p_n)$ to be the MEC of $\{p_1, \dots, p_n\}$, and let $P_i = \{p_1, \dots, p_i\}$.

The algorithm, initially [proposed](#) by Welzl in 1991, goes as follows:

1. Apply a random permutation to the input sequence of points.
2. Maintain the current candidate to be the MEC C , starting with $C = \text{mec}(p_1, p_2)$.
3. Iterate over $i = 3..n$ and check if $p_i \in C$.

- a. If $p_i \in C$ it means that C is the MEC of P_i .
- b. Otherwise, assign $C = \text{mec}(p_i, p_1)$ and iterate over $j = 2..i$ and check if $p_j \in C$.
 - i. If $p_j \in C$, then C is the MEC of P_j among circles that pass through p_i .
 - ii. Otherwise, assign $C = \text{mec}(p_i, p_j)$ and iterate over $k = 1..j$ and check if $p_k \in C$.
 - A. If $p_k \in C$, then C is the MEC of P_k among circles that pass through p_i and p_j .
 - B. Otherwise, $C = \text{mec}(p_i, p_j, p_k)$ is the MEC of P_k among circles that pass through p_i and p_j .

We can see that each level of nestedness here has an invariant to maintain (that C is the MEC among circles that also pass through additionally given 0, 1 or 2 points), and whenever the inner loop closes, its invariant becomes equivalent to the invariant of the current iteration of its parent loop. This, in turn, ensures the *correctness* of the algorithm as a whole.

Omitting some technical details, for now, the whole algorithm can be implemented in C++ as follows:

```
struct point {...};

// Is represented by 2 or 3 points on its circumference
struct mec {...};

bool inside(mec const& C, point p) {
    return ...;
}

// Choose some good generator of randomness for the shuffle
mt19937_64 gen(...);
mec enclosing_circle(vector<point> &p) {
    ranges::shuffle(p, gen);
    auto C = mec{p[0], p[1]};
    for(int i = 0; i < n; i++) {
        if(!inside(C, p[i])) {
            C = mec{p[i], p[0]};
            for(int j = 0; j < i; j++) {
                if(!inside(C, p[j])) {
                    C = mec{p[i], p[j]};
                    for(int k = 0; k < j; k++) {
                        if(!inside(C, p[k])) {
                            C = mec{p[i], p[j], p[k]};
                        }
                    }
                }
            }
        }
    }
    return C;
}
```

Now, it is to be expected that checking that a point p_i is inside the MEC of 2 or 3 points can be done in $O(1)$ (we will discuss this later on). But even then, the algorithm above looks as if it would take $O(n^3)$ in the worst case just because of all the nested loops. So, how come we claimed the linear expected runtime? Let's figure out!

Complexity analysis

For the inner-most loop (over k), clearly its expected runtime is $O(j)$ operations. What about the loop over j ?

It only triggers the next loop if p_j is on the boundary of the MEC of P_j that also passes through point i , and removing p_j would further shrink the circle. Of all points in P_j there can only be at most 2 points with such property, because if there are more than 2 points from P_j on the boundary, it means that after removing any of them, there will still be at least 3 points on the boundary, sufficient to uniquely define the circle.

In other words, after initial random shuffle, there is at most $\frac{2}{j}$ probability that we get one of the at most two unlucky points as p_j . Summing it up over all j from 1 to i , we get the expected runtime of

$$\sum_{j=1}^i \frac{2}{j} \cdot O(j) = O(i).$$

In exactly same fashion we can now also prove that the outermost loop has expected runtime of $O(n)$.

Checking that a point is in the MEC of 2 or 3 points

Let's now figure out the implementation detail of `point` and `mec`. In this problem, it turns out to be particularly useful to use `std::complex` as a class for points:

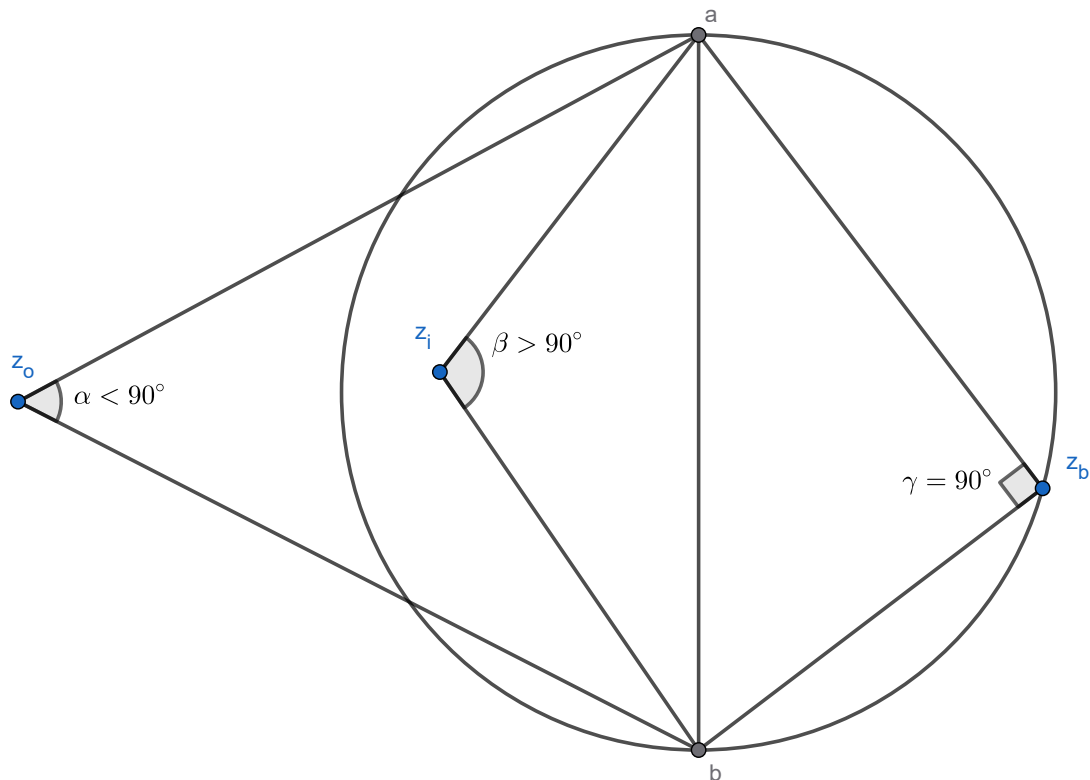
```
using ftype = int64_t;
using point = complex<ftype>;
```

As a reminder, a complex number is a number of type $x + yi$, where $i^2 = -1$ and $x, y \in \mathbb{R}$. In C++, such complex number is represented by a 2-dimensional point (x, y) . Complex numbers already implement basic component-wise linear operations (addition, multiplication by a real number), but also their multiplication and division carry certain geometric meaning.

Without going in too much detail, we will note the most important property for this particular task: Multiplying two complex numbers adds up their polar angles (counted from Ox counter-clockwise), and taking a conjugate (i.e. changing $z = x + yi$ into $\bar{z} = x - yi$) multiplies the polar angle with -1 . This allows us to formulate some very simple criteria for whether a point z is inside the MEC of 2 or 3 specific points.

MEC of 2 points

For 2 points a and b , their MEC is simply the circle centered at $\frac{a+b}{2}$ with the radius $\frac{|a-b|}{2}$, in other words the circle that has ab as a diameter. To check if z is inside this circle we simply need to check that the angle between za and zb is not acute.



Inner angles are obtuse, external angles are acute and angles on the circumference are right

Equivalently, we need to check that

$$I_0 = (b - z)\overline{(a - z)}$$

doesn't have a positive real coordinate (corresponding to points that have a polar angle between -90° and 90°).

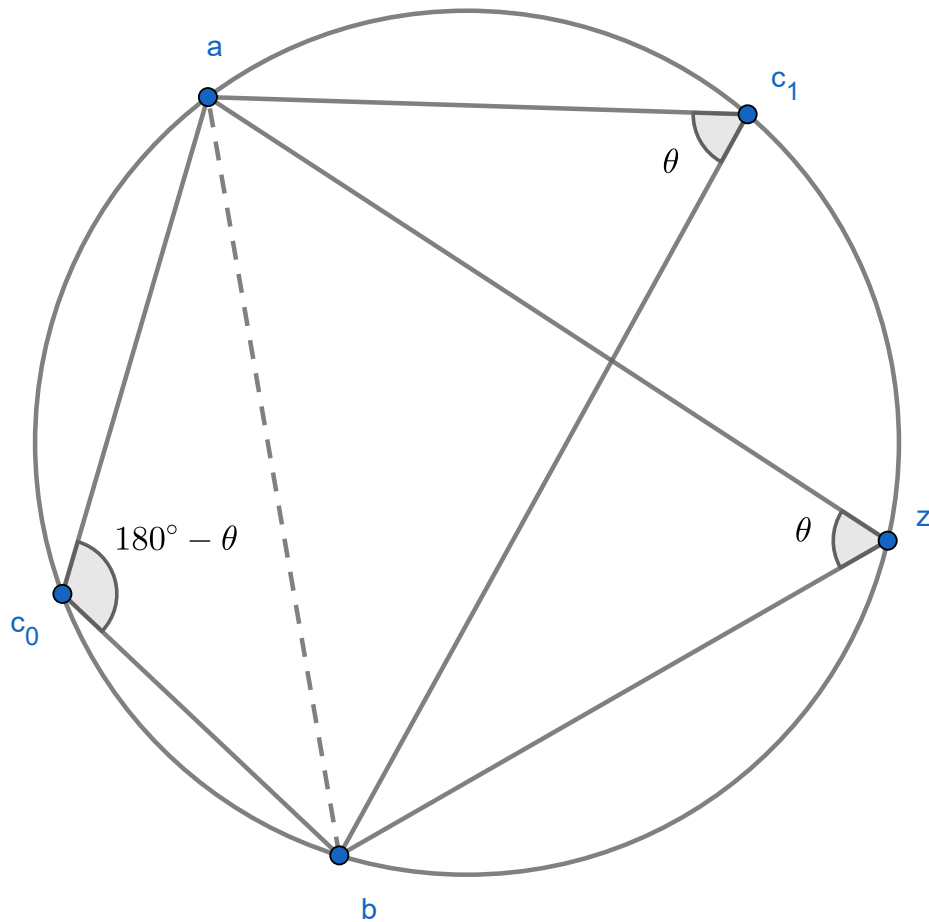
MEC of 3 points

Adding z to the triangle abc will make it a quadrilateral. Consider the following expression:

$$\angle azb + \angle bca$$

In a **cyclic quadrilateral**, if c and z are from the same side of ab , then the angles are equal, and will add up to 0° when summed up signed (i.e. positive if counter-clockwise and negative if clockwise).

Correspondingly, if c and z are on the opposite sides, the angles will add up to 180° .



Adjacent inscribed angles are same, opposing angles complement to 180 degrees

In terms of complex numbers, we can note that $\angle azb$ is the polar angle of $(b - z)\overline{(a - z)}$ and $\angle bca$ is the polar angle of $(a - c)\overline{(b - c)}$. Thus, we can conclude that $\angle azb + \angle bca$ is the polar angle of

$$I_1 = (b - z)\overline{(a - z)}(a - c)\overline{(b - c)}$$

If the angle is 0° or 180° , it means that the imaginary part of I_1 is 0, otherwise we can deduce whether z is inside or outside of the enclosing circle of abc by checking the sign of the imaginary part of I_1 . Positive imaginary part corresponds to positive angles, and negative imaginary part corresponds to negative angles.

But which one of them means that z is inside or outside of the circle? As we already noticed, having z inside the circle generally increases the magnitude of $\angle azb$, while having it outside the circle decreases it. As such, we have the following 4 cases:

1. $\angle bca > 0^\circ$, c on the same side of ab as z . Then, $\angle azb < 0^\circ$, and $\angle azb + \angle bca < 0^\circ$ for points inside the circle.
2. $\angle bca < 0^\circ$, c on the same side of ab as z . Then, $\angle azb > 0^\circ$, and $\angle azb + \angle bca > 0^\circ$ for points inside the circle.

3. $\angle bca > 0^\circ$, c on the opposite side of ab to z . Then, $\angle azb > 0^\circ$ and $\angle azb + \angle bca > 180^\circ$ for points inside the circle.
4. $\angle bca < 0^\circ$, c on the opposite side of ab to z . Then, $\angle azb < 0^\circ$ and $\angle azb + \angle bca < 180^\circ$ for points inside the circle.

In other words, if $\angle bca$ is positive, points inside the circle will have $\angle azb + \angle bca < 0^\circ$, otherwise they will have $\angle azb + \angle bca > 0^\circ$, assuming that we normalize the angles between -180° and 180° . This, in turn, can be checked by the signs of imaginary parts of $I_2 = (a - c)\overline{(b - c)}$ and $I_1 = I_0 I_2$.

Note: As we multiply four complex numbers to get I_1 , the intermediate coefficients can be as large as $O(A^4)$, where A is the largest coordinate magnitude in the input. On the bright side, if the input is integer, both checks above can be done fully in integers.

Implementation

Now, to actually implement the check, we should first decide how to represent the MEC. As our criteria work with the points directly, a natural and efficient way to do this is to say that MEC is directly represented as a pair or triple of points that defines it:

```
using mec = variant<
    array<point, 2>,
    array<point, 3>
>;
```

Now, we can use `std::visit` to efficiently deal with both cases in accordance with criteria above:

```
/* I < 0 if z inside C,
   I > 0 if z outside C,
   I = 0 if z on the circumference of C */
ftype indicator(mec const& C, point z) {
    return visit([&](auto &&C) {
        point a = C[0], b = C[1];
        point I0 = (b - z) * conj(a - z);
        if constexpr (size(C) == 2) {
            return real(I0);
        } else {
            point c = C[2];
            point I2 = (a - c) * conj(b - c);
            point I1 = I0 * I2;
            return imag(I2) < 0 ? -imag(I1) : imag(I1);
        }
    }, C);
}

bool inside(mec const& C, point p) {
    return indicator(C, p) <= 0;
}
```

Now, we can finally ensure that everything works by submitting the problem to the Library Checker: [#308668](#).